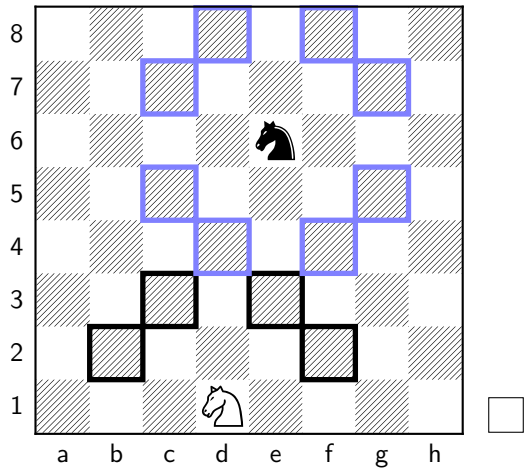
Esercitazione 5: Greedy Algorithms

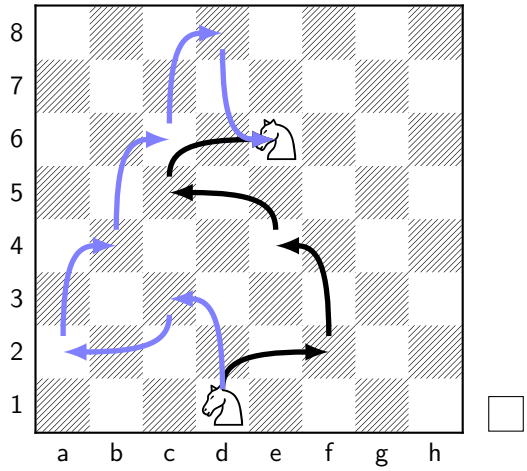
Giacomo Paesani

April 22, 2026

Esercizio 1. Nel gioco degli scacchi il *cavallo* si muove nella seguente maniera: due caselle in orizzontale e una in verticale, o viceversa. Fornire un algoritmo in pseudo-codice che, data una scacchiera con N righe e M colonne, la posizione di partenza e quella di arrivo, calcola il numero minimo di turni necessari ad un cavallo di passare da una posizione all'altra in maniera che il tempo di esecuzione sia pari a $\mathcal{O}(M \cdot N)$.

Che modifiche sono necessarie se invece del cavallo si usa uno degli altri pezzi? Oppure se sono presenti degli altri pezzi statici ma non mangiabili?





Soluzione 1. Prima di risolvere questo esercizio, è necessario trasformare la richiesta in un problema informatico e algoritmico. La scacchiera diventa grafo non diretto con $M \cdot N$ vertici, uno per ogni casella; in oltre, nel caso che il pezzo in considerazione è un cavallo, ogni vertice del grafo ha al più otto vertici adiacenti possibili perché vengono escluse tutte quelle posizioni che risultano *fuori dalla scacchiera*. Allora in seguito alla creazione di questo grafo, la richiesta dell'esercizio diventa quella di trovare il cammino minimo tra due vertici nel grafo.

La soluzione proposta è data dal Algoritmo 1. I vettori R e C sono dati in maniera che, per ogni valore $k = 1, \ldots, 8$, la coppia $(R[k], C[k])$ rappresenta una diversa possibile mossa del cavallo. La matrice $Dist$, inizializzata a 0, rappresenta alla fine dell'esecuzione dell'algoritmo la distanza di ogni casella da quella di partenza. La matrice $Visited$ rappresenta, nel corso dell'esecuzione dell'algoritmo, se una casella è stata già visitata. La funzione isValid, data in Linea 27 con input (M, N, x, y) ha il semplice compito di controllare se il vertice (x, y) è all'interno della scacchiera $M \times N$.

Ora analizziamo la funzione principale ShortestPath-Knight. Questa è una modifica del classico algoritmo di BFS. L'idea è di riconoscere quali sono gli archi di questo grafo che modella la scacchiera mentre si esegue la visita in profondità. Infatti, dato un vertice (x, y) della scacchiera, i suoi vicini sono tutti i vertici della scacchiera della forma $(x + R[k], y + C[k])$, per qualche $k \in \{1, \ldots, 8\}$, cioè quelli raggiungibili da (x, y) con una sola mossa.

Che succede se non esiste nessuna sequenza di mosse per muovere il cavallo dalla casella iniziale (a, b) ad un altra casella (x, y)? Cioè come valutiamo

Algorithm 1

Input: quattro numeri interi M, N, a e b.
Output: vettore $Dist$ delle distanze dalla casella di cordinate (a, b).

```
global variables
    R ↤ vettore di 8 elementi, inizializzato come {2, 2, −2, −2, 1, 1, −1, −1}
    C ↤ vettore di 8 elementi, inizializzato come {−1, 1, 1, −1, 2, −2, 2, −2}
    Dist ↤ matrice M × N, inizializzata a 0
    Visited ↤ matrice M × N, inizializzata a 0
end global variables
function ShortestPath-Knight(M, N, a, b)
    if ¬(1 ≤ a ≤ M and 1 ≤ b ≤ N) then
        for x ∈ {1, ..., M} and y ∈ {1, ..., N} do
            Dist[(x, y)] = +∞
        return Dist
    Visited[(a, b)] = 1
    Q.enqueue(a, b)
    while Q ≠ ∅ do
        (x, y) ↤ Q.dequeue()
        for k ∈ {1, ..., 8} do
            x′ = x + R[k]
            y′ = y + C[k]
            if isValid(M,N,x′,y′)=TRUE and Visited[(x′, y′)] = 0 then
                Visited[(x′, y′)] = 1
                Dist[(x′, y′)] = Dist[(x, y)] + 1
                Q.enqueue(w)
    for x ∈ {1, ..., M} and y ∈ {1, ..., N} do
        if Visited[(x, y)] = 0 then
            Dist[(x, y)] = +∞
    return Dist
function isValid(M,N,x,y)
    if 1 ≤ x ≤ M and 1 ≤ y ≤ N then
        return TRUE
    return FALSE
```

rapidamente se (a, b) e (x, y) sono in diverse componenti connesse del grafo della scacchiera? Se tale condizione è vera, allora esistono due vertici u e v tali che a $Dist[u, v]$ viene assegnato il valore $+\infty$.

Notiamo che il grafo creato ha $M \cdot N$ vertici. Dato che ogni vertice ha grado massimo 8, per il lemma delle strette di mano (Handshaking's Lemma) si ha che ci sono al più $16M \cdot N$ archi. Quindi la complessità dell'algoritmo proposto è $\mathcal{O}(M \cdot N)$. $\square$

Esercizio 2. Consideriamo il problema di trovare un sottoinsieme di cardinalità massima di intervalli disgiunti da un insieme qualsiasi di intervalli $\{a_i = [s_i, f_i]\}$. Un algoritmo greedy che produce una soluzione ottima consiste nel selezionare ad ogni passo un intervallo, tra quelli che sono disgiunti con quelli precedentemente scelti, che (0) ha l'estremo destro di valore minimo.

Consideriamo alternative idee per un algoritmo greedy per lo stresso problema. Selezionare ad ogni passo un intervallo, tra quelli che sono disgiunti con quelli precedentemente scelti, che:

(1) ha l'estremo sinistro di valore massimo.

(2) ha o l'estremo destro di valore minimo o l'estremo sinistro di valore massimo.

(3) ha ampiezza $f_i - s_i$ minima.

(4) interseca il numero minimo di intervalli rimanenti.

(5) ha l'estremo sinistro di valore minimo.

Per ogniuna di queste soluzioni proposte dimostrare che esse producono sempre soluzioni ottime o fornire un contro-esempio in cui la soluzione ottenuta non è ottima.

Soluzione 2. La strategia (1) è sostanzialmente equivalente alla strategia (0). Una maniera per vedere questo fatto è quella di cambiare segno a tutti gli intervalli, cioè invece di considerare il problema P con input $a_i = [s_i, f_i]$, consideriamo l'equivalente problema P' con input $b_i = [-f_i, -s_i]$. In breve, dire che due problemi P e P' sono equivalenti significa che se so risolvere il problema P con una certa complessità allora so anche risolvere il problema P' *trasformando* la soluzione per P con un costo computazionale *basso*. Allora,

adottare la strategia (0) al problema P', equivale ad adottare la strategia (1) al problema P. La strategia (2) è una strategia ibrida tra quella proposta nel testo dell'esercizio e la (1), quindi ogni soluzione ottenuta adottando la strategia (2) è ottima.

La strategia (3) non produce sempre soluzioni ottime e per affermare questo è sufficiente mostrare un esempio. Sia $a_1 = [1, 5]$, $a_2 = [4, 7]$ e $a_3 = [6, 12]$. Le ampiezze di a_1, a_2 e a_3 sono rispettivamente di 4, 3 e 6 quindi la strategia (3) produce come soluzione l'insieme $\{a_2\}$ mentre la soluzione ottima è $\{a_1, a_3\}$.

Per la strategia (4), il discorso è similare. Sia $a_1 = [1, 3]$, $a_2 = [2, 5]$, $a_3 = [4, 7]$, $a_4 = [6, 9]$ e $a_5 = [8, 10]$. Ogniuno degli intervalli a_2, a_3 e a_4 interseca altri due intervalli, in particolare a_2 interseca a_1 e a_3, a_3 interseca a_2 e a_4 e infine a_3 interseca a_2 e a_5. Se quindi al primo passo scelgo a_2 allora ottengo o la soluzione $\{a_2, a_4\}$ o $\{a_2, a_5\}$ ed entrambe hanno meno elementi della soluzione ottima $\{a_1, a_3, a_5\}$.

Anche la strategia (5) non produce sempre soluzioni ottime. Infatti, sia $a_1 = [1, 6]$, $a_2 = [2, 3]$ e $a_3 = [4, 5]$. Dato che l'intervallo a_1 è quello di estremo sinistro minimo, allora la soluzione ottenuta adottando la strategia (5) è $\{a_1\}$ quando invece quella ottima è $\{a_2, a_3\}$. $\square$

Esercizio 3. Sia A un insieme, una *partizione* di A è una collezione $\{A_i\}$ di sottoinsiemi di A tali che $A_i \cap A_j = \emptyset$ per ogni $i \neq j$ e $\bigcup A_i = A$. Fornire un algoritmo in pseudo-codice che, data una collezione di intervalli $A = \{a_i = [s_i, f_i]\}$, determina il numero minimo di sottoinsiemi $A_i \subseteq A$ che formano una partizione di A tali che in ogni sottoinsieme A_i gli intervalli sono due a due disgiunti. Tempo di esecuzione $\mathcal{O}(|A|^2)$.

Soluzione 3. Una strategia possibile sarebbe quella di usare l'algoritmo greedy descritto nel Esercizio 2 nella seguente maniera. L'insieme A_1 è un insieme di cardinalità massima di A in cui gli intervalli sono a due a due disgiunti. Più in generale si può pensare che l'insieme A_{k+1} è un sottoinsieme di cardinalità massima di $A \setminus (A_1 \cup \ldots \cup A_k)$ in cui gli intervalli sono due a due disgiunti. Questo procedimento continua finché l'insieme $A \setminus (A_1 \cup \ldots \cup A_k)$ è non vuoto. Usare una strategia greedy in questo caso non porta sempre ad una soluzione otttima e ciò è evidenziato dal seguente contro-esempio. Sia $a_1 = [0, 4]$, $a_2 = [1, 2]$, $a_3 = [3, 9]$, $a_4 = [5, 6]$ e $a_5 = [7, 8]$. Allora usando la strategia proprosta otteniamo $A_1 = \{a_2, a_4, a_5\}$, $A_2 = \{a_1\}$ e $A_3 = \{a_3\}$ quando la soluzione ottima è data da $A_1 = \{a_1, a_4, a_5\}$, $A_2 = \{a_2, a_3\}$.

Una soluzione corretta è quella descritta nel Algoritmo 2. L'idea è di costruire contemporaneamente gli insiemi della partizione in maniera greedy: un intervallo con estremo destro di valore minimo tra quelli in cui non sono stati messi ancora in alcun elemento della partizione viene messo, se esiste, nell'elemento della partizione con indice minimo in cui è disgiunto con tutti gli altri intervalli di quel elemento o un nuovo elemento della partizione viene creato.

Algorithm 2 Algoritmo per partizionare un insieme di intervalli in maniera che in ogni elemento gli intervalli siano compatibili.

Input: Un insieme di intervalli A, ordinati con estremo destro non-decrescente.
Output: numero minimo di elementi compatibili di una partizione di A.

```
 1: function IntervalColoring(A)
 2:     n = |A|
 3:     if n == 0 then
 4:         return 0
 5:     k = 1
 6:     for i = 1, ..., n do
 7:         t = 0
 8:         for j = 1, ..., k do
 9:             if isCompatible(A_j,a_i)==TRUE and t == 0 then
10:                 A_j.Add(a_i)
11:                 t = 1
12:         if t == 0 then
13:             k = k + 1
14:             A_k.Add(a_i)
15:     return k
16: function isCompatible(A,a*)
17:     for a ∈ A do
18:         if a ∩ a* ≠ ∅ then
19:             return FALSE
20:     return TRUE
```

Prima analiziamo la funzione **isCompatible**: essa prende in input un insieme di intervalli A e un intervallo a^* e controlla se a^* è disgiunto da ogni elemento di A. Nel caso questo non è vero, allora l'algoritmo ritorna `FALSE` (Linea 19) e ritorna `TRUE` altrimenti (Linea 20).

La funzione principale di questa soluzione è **IntervalColoring**: il nome deriva dal fatto che questo problema equivale al problema di colorare i ver-

tici ristretta ad una classe di grafi chiamati *interval*. Se A è l'insieme vuoto, allora l'unica partizione di A è quella senza elementi: questo caso è coperto dal controllo **if** in Linea 3. Allora consideriamo il caso in cui A è non vuoto e supponiamo che gli elementi di A sono ordinati in ordine crescente in base al valore dell'estremo destro. Con il ciclo **for** di Linea 8, cerchiamo se l'intervallo a_i che vorrei inserire è compatibile con almeno uno degli insiemi di intervalli già creati, $A_1, \ldots, A_k$: se tale insieme esiste, allora aggiungo a_i solo all'insieme con cui è compatibile di indice minimo. Altrimenti, cioè se a_i non è compatibile con nessun insieme precedentemente creato, necessariamente dovremmo creare un nuovo insieme A_{k+1} dove poter inserire a_i (controllo **if** in Linea 12). La variabile t e i controlli su essa servono ad assicurarsi che ogni intervallo di A viene inserito in esattamente un elemento della partizione. $\square$